

SQL Assignment — Group 7: PLSQL Blocks

Group 7 — PL/SQL Blocks (Oracle)

1. Declare variables + DBMS_OUTPUT

Write a PL/SQL block that declares two variables — a user's name (VARCHAR2) and age (NUMBER) — assigns values, and prints both using DBMS_OUTPUT.PUT_LINE.

```
SET SERVEROUTPUT ON;

DECLARE
    v_name VARCHAR2(50);
    v_age  NUMBER;
BEGIN
    v_name := 'Aisha Khan';
    v_age  := 27;

    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Age: '  || v_age);
END;
/
```

Explanation: The DECLARE section defines two variables with their types (VARCHAR2 for text, NUMBER for numeric values). Inside the BEGIN...END block, values are assigned using :=, and DBMS_OUTPUT.PUT_LINE prints text to the console, with || used to concatenate the label string with the variable's value (implicitly converting the number to text).

2. IF statement — free delivery eligibility

Write a PL/SQL block using an IF statement to check if an order amount is above 500. If yes, print 'Eligible for free delivery', else print 'Delivery charges apply'.

```
SET SERVEROUTPUT ON;

DECLARE
    v_order_amount NUMBER := 650;
BEGIN
    IF v_order_amount > 500 THEN
        DBMS_OUTPUT.PUT_LINE('Eligible for free delivery');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Delivery charges apply');
    END IF;
END;
/
```

Explanation: The variable is initialized directly in its declaration (:= 650). The IF ... THEN ... ELSE ... END IF; structure branches based on whether v_order_amount exceeds 500, printing the appropriate message for each case.

3. LOOP printing numbers 1 to 5

Write a PL/SQL block that uses a simple LOOP to print the numbers 1 to 5.

```

SET SERVEROUTPUT ON;

DECLARE
    v_counter NUMBER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(v_counter);
        v_counter := v_counter + 1;
        EXIT WHEN v_counter > 5;
    END LOOP;
END;
/

```

Explanation: A basic `LOOP ... END LOOP;` repeats indefinitely until explicitly stopped. Each iteration prints the current `v_counter` value, increments it by 1, then checks `EXIT WHEN v_counter > 5` — once the counter passes 5, the loop exits, so the numbers 1 through 5 are printed exactly once each.

4. Cursor over PRODUCTS table

Declare a cursor to select all product names from a table called PRODUCTS. Fetch each product name and print it.

```

SET SERVEROUTPUT ON;

DECLARE
    CURSOR product_cursor IS
        SELECT product_name FROM PRODUCTS;
    v_product_name PRODUCTS.product_name%TYPE;
BEGIN
    OPEN product_cursor;
    LOOP
        FETCH product_cursor INTO v_product_name;
        EXIT WHEN product_cursor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_product_name);
    END LOOP;
    CLOSE product_cursor;
END;
/

```

Explanation: A CURSOR is declared to hold the result set of `SELECT product_name FROM PRODUCTS`. After OPEN-ing it, the loop repeatedly FETCHes one row at a time into `v_product_name`; `%TYPE` makes this variable automatically match the column's data type. `EXIT WHEN product_cursor%NOTFOUND` stops the loop once there are no more rows to fetch, and `CLOSE` releases the cursor's resources afterward.

5. Exception handling — divide by zero

Write a PL/SQL block that divides two numbers. If a division by zero occurs, catch the exception and print 'Cannot divide by zero'. Otherwise, print the result.

```

SET SERVEROUTPUT ON;

DECLARE
    v_num1 NUMBER := 10;
    v_num2 NUMBER := 0;
    v_result NUMBER;
BEGIN
    v_result := v_num1 / v_num2;
    DBMS_OUTPUT.PUT_LINE('Result: ' || v_result);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');
END;
/

```

Explanation: The division is attempted inside the `BEGIN` block as normal. Oracle raises the predefined `ZERO_DIVIDE` exception automatically whenever a division by zero is attempted, and the `EXCEPTION WHEN ZERO_DIVIDE THEN ...` handler catches it and prints the friendly message instead of letting the program crash with an unhandled error. If `v_num2` were non-zero, execution would never reach the exception handler and the result would print normally.